Based on the provided notes for Module 1 and Module 2, here are the solutions for the first internal evaluation test (CIE Test 1) up to question 6b.

Question 1a

List the various steps and strategies to build logical solutions to problems and suggest an appropriate approach to analyze and resolve the problem of long customer wait time in a call center ensuring enhanced service levels.

Answer: The steps and strategies to build logical solutions are:

- **Understand the Problem:** Define the problem scope, gather relevant data, and identify constraints.
- **Decompose the Problem:** Break it into smaller, manageable sub-problems.
- **Analyze the Causes:** Use root cause analysis to determine underlying factors.
- **Identify Goals and Objectives:** Clearly state desired outcomes and success criteria.
- **Generate Ideas and Solutions:** Brainstorm and prioritize solutions.
- **Evaluate Alternatives:** Assess risks, costs, and benefits.
- **Select the Best Solution:** Choose the one aligning best with goals.
- **Plan Implementation:** Develop a detailed execution plan.
- **Execute and Monitor:** Implement the solution and monitor effectiveness.

Approach for Call Center Wait Times: Following the "Healthcare" example logic, the approach would be:

- **Analysis:** Analyze customer flow, staffing levels, and routing processes to identify bottlenecks.
- **Solutions:** Optimize staff scheduling, implement automated response systems (like IVR) for basic queries, and prioritize urgent calls to reduce wait times.

Question 1b

Explain commonly used flowchart symbols along with their purposes. Design a flowchart to find: (i) If the given number is even or odd (ii) Area of Rectangle.

Answer: Common flowchart symbols include:

- **Terminal (Oval):** Indicates the start or end of a program.
- **Input/Output (Parallelogram):** Used for obtaining data or outputting results.
- **Process (Rectangle):** Indicates internal operations like calculations.
- **Decision (Diamond):** Used for questions with binary (Yes/No) answers.
- **Flow Lines (Arrows):** Show the direction of the process flow.

Flowchart Designs:

- **(i) Even or Odd:** Start $\rightarrow$ Input num $\rightarrow$ Decision (is num % 2 == 0?) $\rightarrow$ If Yes, print "Even"; If No, print "Odd" $\rightarrow$ Stop.
- **(ii) Area of Rectangle:** Start $\rightarrow$ Input length, width $\rightarrow$ Process (Area = length * width) $\rightarrow$ Output Area $\rightarrow$ Stop.

Question 2a

Define Hardware & Software. Categorize the following into system software and application software. (i) Mac OS (ii) MATLAB (iii) VLC media player (iv) Apache HTTP server (v) GCC compiler (vi) Facebook.

Answer:

- **Hardware:** The physical parts of a computer that you can touch, such as the CPU, monitor, keyboard, and mouse.
- **Software:** A set of instructions, data, or programs that tell the computer what to do.

Categorization:

- **System Software:** Designed for running computer hardware and providing a platform for application programs.
  - (i) Mac OS (Operating System)

    - (iv) Apache HTTP server (Server software)
    - (v) GCC compiler (Compiler)

- **Application Software:** Designed for users to perform specific tasks.

    - (ii) MATLAB (Technical computing)
    - (iii) VLC media player (Media playback)

    - (vi) Facebook (Social media/Web application)

Question 2b

Explain any four characteristics of algorithms. Design an algorithm to swap or exchange two numbers using the third variable.

Answer: Four characteristics of algorithms are:

1. **Unambiguous:** Every step must be clearly defined and understood with no room for interpretation.

2. **Termination:** The algorithm must eventually reach a conclusion and not run indefinitely.

3. **Input and Output:** It takes input data and produces a result based on logic.

4. **Deterministic:** For a given input, it will always produce the same output.

Algorithm to Swap Two Numbers (using a third variable):

1. Start.

2. Read values for variables a and b.

3. Assign value of a to a temporary variable $temp$ ($temp = a$).

4. Assign value of b to a ($a = b$).

5. Assign value of $temp$ to b ($b = temp$).

6. Print the swapped values of a and b.

7. Stop.

Question 3a

Evaluate the followings step by step and write the output.

Answer:

- **Expression 1: 200/40 <= 20 - 10 + 200%20 - 40 == 10 >= 1 != 40**
    1. Arithmetic first: $200/40 = 5$; $200\%20 = 0$.
    2. $5 <= 20 - 10 + 0 - 40 \rightarrow 5 <= -30$.
    3. $5 <= -30$ is false (0).
    4. $0 == 10 \rightarrow 0$ (false).
    5. $0 >= 1 \rightarrow 0$ (false).
    6. $0 != 40 \rightarrow 1$ (true).

    - **Output: 1**
- **Expression 2: x+=y*=z-=4 (Given x=6, y=10, z=9)**
    - Evaluates right to left.

    1. $z = 9 - 4 = 5$.
    2. $y = 10 * 5 = 50$.
    3. $x = 6 + 50 = 56$.
    - **Output: 56**
- **Expression 3: 2*((a%5)*(4+(b-3)/(c+2))) (Given a=8, b=15, c=4)**
    1. $a\%5 = 8\%5 = 3$.
    2. $b-3 = 15-3 = 12$; $c+2 = 4+2 = 6$.
    3. $12/6 = 2$.
    4. $4+2 = 6$.
    5. $3 * 6 = 18$.
    6. $2 * 18 = 36$.

    - **Output: 36**
- **Expression 4: 15 - b++ + --c - 8/4 * 2 - ++a (Given a=5, b=6, c=3)**
    1. Unary first: ++a becomes 6; --c becomes 2.

    2. Post-increment b++ uses current value 6, then becomes 7.
    3. Arithmetic: $8/4*2 = 2*2 = 4$.
    4. $15 - 6 + 2 - 4 - 6 = 1$.
    - **Output: 1**

Question 3b

Explain the basic structure of C program. Construct a C program to check whether a person is eligible to get driving licence or not.

Answer: The basic structure includes:

- **Documentation Section:** Comments about the program.
- **Link Section:** Preprocessor directives like #include <stdio.h>.
- **Definition Section:** Symbolic constants like #define PI 3.14.
- **Global Declaration:** Variables used across multiple functions.
- **main() function:** The entry point where execution begins.
- **Subprogram Section:** User-defined functions.

C Program for Driving License Eligibility:

C

```c
#include <stdio.h>
void main() {
  int age;
  printf("Enter your age: ");
  scanf("%d", &age);
  if(age >= 18) {
    printf("You are eligible for a driving license.");
  } else {
    printf("You are not eligible.");
  }
}
```

Question 4a

State any four differences between a while loop and a do-while loop. Develop a C

Program to count the digits in a given number using do-while loop.

Answer:

Feature	**while loop**	**do-while loop**
Control	Entry-controlled: condition checked first.	Exit-controlled: condition checked at the end.
Execution	May not execute at all if condition is false.	Executes at least once regardless of condition.
Semicolon	No semicolon after while condition.	Semicolon (;) is required after while condition.
Use	Used when iterations are unknown.	Used when the body must run at least once.

C Program to Count Digits:

C

```
#include <stdio.h>
void main() {
  int num, count = 0;
  printf("Enter a number: ");
  scanf("%d", &num);
  do {
    num /= 10;
    count++;
  } while(num != 0);
  printf("Number of digits: %d", count);
```

}

Question 4b

Discuss the following operators with an example: (i) Pre Increment Operator (ii) Post Decrement Operator (iii) Bitwise & operator.

Answer:

- **(i) Pre Increment (++a):** Increases the value of the operand by 1 before it is used in the expression. Example: If a=10, b=++a results in both a=11 and b=11.

- **(ii) Post Decrement (a--):** Uses the current value in the expression first, then decreases it by 1. Example: If a=10, b=a-- results in b=10 and a=9.

- **(iii) Bitwise & (AND):** Performs bit-by-bit AND operation. Example: 12 & 25 (Binary: 1100 & 11001) results in 8 (Binary: 1000).

Question 5a

Explain the working of switch statement with syntax. Develop a C program to simulate simple calculator using switch statement.

Answer: The switch statement is a multi-way branch statement that selects one option out of several based on the value of an expression.

Syntax:

C

```c
switch(expression) {
  case constant1: statements; break;
  case constant2: statements; break;
  default: default_statements;
}
```

Calculator Program:

C

```
#include <stdio.h>
void main() {
  char op;
  double n1, n2;
  printf("Enter operator (+, -, *): ");
  scanf(" %c", &op);
  printf("Enter two operands: ");
  scanf("%lf %lf", &n1, &n2);
  switch(op) {
    case '+': printf("%.1lf + %.1lf = %.1lf", n1, n2, n1+n2); break;
    case '-': printf("%.1lf - %.1lf = %.1lf", n1, n2, n1-n2); break;
    case '*': printf("%.1lf * %.1lf = %.1lf", n1, n2, n1*n2); break;
    default: printf("Error! operator is not correct");
  }
}
```

Question 5b

Write the general syntax of for loop and explain. Construct a C program to find the factorial of a given number using for loop.

Answer: The for loop is used for repeating a block of code a fixed number of times.

Syntax: for (initialization; condition; increment/decrement) { statements; }

- **Initialization:** Executed once at the start.
- **Condition:** Evaluated before every iteration; loop stops if false.
- **Update:** Executed after every iteration to change the counter.

Factorial Program:

C

```c
#include <stdio.h>
void main() {
  int i, n, fact = 1;
  printf("Enter a number: ");
  scanf("%d", &n);
  for(i = 1; i <= n; i++) {
    fact = fact * i;
  }
  printf("Factorial: %d", fact);
}
```

Question 6a

Write a C program to print the following pattern using nested for loop.

(i) * (ii) 5 5 5 5 5 ...

Answer (Pattern i):

C

```c
#include <stdio.h>
void main() {
  int i, j;
  for(i = 1; i <= 5; i++) {
    for(j = 1; j <= i; j++) {
      printf("* ");
    }
    printf("\n");
  }
```

}

Question 6b

Write the output of the following programs.

Answer:

- **(i)** for(i=1; i<=10; i++) { if(i==6) break; printf("%d\t", i); }
  - **Output: 1 2 3 4 5**

- **(ii)** for(i=1; i<=10; i++) { if(i%5==0) continue; printf("%d\t", i); }
  - **Output: 1 2 3 4 6 7 8 9**

- **(iii)** printf("MITE\t"); goto down; printf("IS NOT IN"); down: printf("MOODABIDRI");
  - **Output: MITE MOODABIDRI**

- **(iv)** switch(a) { case 3: printf("I am 3"); break; case 5: printf("I am 5"); break; } (Given a=3)
  - **Output: I am 3**

[2026-04-17 1:38 PM IST]

Based on the provided notes for Module 1 and Module 2, here are the solutions for the second question paper (CTP_1_IA.docx) up to question 6b.

Question 1a

Define Computational Thinking and explain its key techniques. Considering the problem of optimizing bus routes for a city, describe how each of these techniques can be applied to solve the problem efficiently.

Answer:

Computational Thinking is a problem-solving process that involves formulating a problem and expressing its solution in a way that a computer can effectively carry out.

Key Techniques applied to Bus Route Optimization:

- **Decomposition:** Breaking the city into smaller zones or sectors and analyzing the travel demand within each zone separately.

- **Pattern Recognition:** Identifying peak travel hours (e.g., morning and evening rushes) and common destinations (e.g., business districts, schools) by looking at historical data.
- **Abstraction:** Focusing on essential information like passenger count and stop locations while ignoring irrelevant details like the color of the bus or the driver's name.
- **Algorithm Design:** Creating a step-by-step set of rules to determine the fastest path for buses based on traffic data and passenger demand.

Question 1b

Define an Algorithm? Design an algorithm and flowchart to convert the temperature from Celsius to Fahrenheit.

Answer:

An **Algorithm** is a step-by-step procedure or a set of rules to be followed in calculations or other problem-solving operations.

Algorithm for Temperature Conversion:

1. Start.
2. Input temperature in Celsius (C).
3. Calculate Fahrenheit (F) using the formula: $F = (C \times 9/5) + 32$.
4. Print the value of F.
5. Stop.

Question 2a

Differentiate between Compiler and Interpreter. Explain the different types of programming languages.

Answer:

Feature	**Compiler**	**Interpreter**
Process	Translates the entire program at once.	Translates program line-by-line.

Execution	Faster execution after compilation.	Slower execution as translation happens during runtime.
Errors	Displays all errors after scanning the whole code.	Stops at the first error encountered.
Example	C, C++, Java.	Python, Ruby, PHP.

Types of Programming Languages:

- **Low-Level Language:** Includes Machine Language (binary) and Assembly Language (using mnemonics). It is hardware-dependent and efficient but hard for humans to read.
- **High-Level Language:** User-friendly languages like C, Python, and Java. They are portable and require a compiler or interpreter to run.

Question 2b

Design an algorithm and flowchart to find the largest of three numbers.

Answer:

Algorithm:

1. Start.
2. Read three numbers A, B, and C.
3. If $A > B$ and $A > C$, then print "A is largest".
4. Else if $B > C$, then print "B is largest".
5. Else, print "C is largest".
6. Stop.

Question 3a

Develop a C program to read two numbers and perform all arithmetic operations (addition, subtraction, multiplication, division, and modulus) and display the results.

Answer:

```c
C

#include <stdio.h>
void main() {
    int a, b;
    printf("Enter two integers: ");
    scanf("%d %d", &a, &b);
    printf("Sum: %d\n", a + b);
    printf("Difference: %d\n", a - b);
    printf("Product: %d\n", a * b);
    if(b != 0) {
        printf("Quotient: %d\n", a / b);
        printf("Modulus: %d\n", a % b);
    } else {
        printf("Division by zero error.");
    }
}
```

Question 3b

Explain the rules for declaring variables in C. Also, identify whether the following variable declarations are valid or invalid.

Answer:

Rules for Variable Declaration:

1. Must begin with a letter or an underscore (_).
2. Cannot start with a digit.

Can contain letters, digits, and underscores.

3. Keywords (like `int`, `float`) cannot be used as variable names.
4. C is case-sensitive (`age` and `AGE` are different).

Validation:

- `int student1;` — **Valid** (Contains letters and numbers).
- `float total marks;` — **Invalid** (Contains a space).
- `char _name;` — **Valid** (Starts with an underscore).
- `double 2amount;` — **Invalid** (Starts with a digit).

Question 4a

Explain the Nested for loop with syntax. Write a C Program to print the following pattern using Nested for loop.

(1) *, **, *** ... (2) 5 5 5 5 5, 4 4 4 4 ...

Answer:

Nested for loop: A loop within another loop. The inner loop completes all its iterations for every single iteration of the outer loop.

Syntax:

```c
for (initialization; condition; increment) {
    for (initialization; condition; increment) {
        // statements
    }
}
```

C Program for Pattern 2 (Numbers):

```c
#include <stdio.h>
void main() {
    int i, j;
    for(i = 5; i >= 1; i--) {
        for(j = 1; j <= i; j++) {
            printf("%d ", i);
        }
        printf("\n");
    }
}
```

Question 4b

If a = 8, b = 15, and c = 4, evaluate the following expression:

(1) 2 * ((a % 5) * (4 + (b – 3) / (c + 2)))

(2) a += b *= c -= 5 (Given a=3, b=5, c=8)

Answer:

- **Expression 1:**
  - $a\%5 = 8\%5 = 3$
  - $b-3 = 12$; $c+2 = 6$
  - $12 / 6 = 2$
  - $4 + 2 = 6$
  - $3 * 6 = 18$
  - $2 * 18 = 36$
  - **Result: 36**
- **Expression 2 (a=3, b=5, c=8):**
  - Evaluates right to left:
  - $c = 8 - 5 = 3$
  - $b = 5 * 3 = 15$
  - $a = 3 + 15 = 18$
  - **Result: a=18, b=15, c=3**

Question 5a

Explain for loop with syntax. Develop a C program to generate the Fibonacci Series using for loop.

Answer:

Syntax: for(initialization; condition; increment/decrement) { statements; }

Fibonacci Program:

C

```c
#include <stdio.h>
void main() {
    int n, i, f1 = 0, f2 = 1, f3;
    printf("Enter number of terms: ");
    scanf("%d", &n);
    printf("Series: %d %d ", f1, f2);
    for(i = 3; i <= n; i++) {
        f3 = f1 + f2;
        printf("%d ", f3);
        f1 = f2;
        f2 = f3;
    }
}
```

Question 5b

Differentiate between while loop and do-while loop with its syntax. Write a C program using a while loop to find the factorial of a given number.

Answer:

while loop	do-while loop
while(condition) { ... }	do { ... } while(condition);
Entry-controlled.	Exit-controlled.

Factorial using while loop:

```
C

#include <stdio.h>
void main() {
    int n, fact = 1;
    printf("Enter a number: ");
    scanf("%d", &n);
    int i = n;
    while(i > 0) {
        fact *= i;
        i--;
    }
    printf("Factorial: %d", fact);
}
```

Question 6a

Develop a C program to find the reverse of a positive integer and check for Palindrome.

Answer:

```
C

#include <stdio.h>
```

```
void main() {
    int n, reversed = 0, original, remainder;
    printf("Enter an integer: ");
    scanf("%d", &n);
    original = n;
    while (n != 0) {
        remainder = n % 10;
        reversed = reversed * 10 + remainder;
        n /= 10;
    }
    if (original == reversed)
        printf("%d is a palindrome.", original);
    else
        printf("%d is not.", original);
}
```

Question 6b

Explain any two Unconditional control transfer statements with syntax and example.

Answer:

1. **break:** Used to exit a loop or switch statement immediately.
   - **Syntax:** `break;`
   - **Example:** `for(i=1; i<=10; i++) { if(i==5) break; }` (stops loop when i is 5).
2. **continue:** Skips the current iteration of a loop and moves to the next one.
   - **Syntax:** `continue;`
   - **Example:** `for(i=1; i<=5; i++) { if(i==3) continue; printf("%d", i); }` (Prints 1, 2, 4, 5).

[2026-04-17 1:39 PM IST]

Based on the provided notes for Module 1 and Module 2, here are the solutions for the second question paper (CTP_1_IA.docx) up to question 6b.

Question 1a

Define Computational Thinking and explain its key techniques. Considering the problem of optimizing bus routes for a city, describe how each of these techniques can be applied to solve the problem efficiently.

Answer:

Computational Thinking is a problem-solving process that involves formulating a problem and expressing its solution in a way that a computer can effectively carry out.

Key Techniques applied to Bus Route Optimization:

- **Decomposition:** Breaking the city into smaller zones or sectors and analyzing the travel demand within each zone separately.
- **Pattern Recognition:** Identifying peak travel hours (e.g., morning and evening rushes) and common destinations (e.g., business districts, schools) by looking at historical data.
- **Abstraction:** Focusing on essential information like passenger count and stop locations while ignoring irrelevant details like the color of the bus or the driver's name.
- **Algorithm Design:** Creating a step-by-step set of rules to determine the fastest path for buses based on traffic data and passenger demand.

Question 1b

Define an Algorithm? Design an algorithm and flowchart to convert the temperature from Celsius to Fahrenheit.

Answer:

An **Algorithm** is a step-by-step procedure or a set of rules to be followed in calculations or other problem-solving operations.

Algorithm for Temperature Conversion:

1. Start.
2. Input temperature in Celsius (C).
3. Calculate Fahrenheit (F) using the formula: $F = (C \times 9/5) + 32$.
4. Print the value of F.
5. Stop.

Question 2a

Differentiate between Compiler and Interpreter. Explain the different types of programming languages.

Answer:

Feature	Compiler	Interpreter

Process	Translates the entire program at once.	Translates program line-by-line.
Execution	Faster execution after compilation.	Slower execution as translation happens during runtime.
Errors	Displays all errors after scanning the whole code.	Stops at the first error encountered.
Example	C, C++, Java.	Python, Ruby, PHP.

Types of Programming Languages:

- **Low-Level Language:** Includes Machine Language (binary) and Assembly Language (using mnemonics). It is hardware-dependent and efficient but hard for humans to read.
- **High-Level Language:** User-friendly languages like C, Python, and Java. They are portable and require a compiler or interpreter to run.

Question 2b

Design an algorithm and flowchart to find the largest of three numbers.

Answer:

Algorithm:

1. Start.
2. Read three numbers A, B, and C.
3. If $A > B$ and $A > C$, then print "A is largest".
4. Else if $B > C$, then print "B is largest".
5. Else, print "C is largest".
6. Stop.

Question 3a

Develop a C program to read two numbers and perform all arithmetic operations (addition, subtraction, multiplication, division, and modulus) and display the results.

Answer:

C

```
#include <stdio.h>
void main() {
    int a, b;
    printf("Enter two integers: ");
    scanf("%d %d", &a, &b);
    printf("Sum: %d\n", a + b);
    printf("Difference: %d\n", a - b);
    printf("Product: %d\n", a * b);
    if(b != 0) {
        printf("Quotient: %d\n", a / b);
        printf("Modulus: %d\n", a % b);
    } else {
        printf("Division by zero error.");
    }
}
```

Question 3b

Explain the rules for declaring variables in C. Also, identify whether the following variable declarations are valid or invalid.

Answer:

Rules for Variable Declaration:

1. Must begin with a letter or an underscore (_).
2. Cannot start with a digit.
3. Can contain letters, digits, and underscores.
4. Keywords (like `int`, `float`) cannot be used as variable names.
5. C is case-sensitive (`age` and `AGE` are different).

Validation:

- `int student1;` — **Valid** (Contains letters and numbers).
- `float total marks;` — **Invalid** (Contains a space).
- `char _name;` — **Valid** (Starts with an underscore).

- double 2amount; — **Invalid** (Starts with a digit).

Question 4a

Explain the Nested for loop with syntax. Write a C Program to print the following pattern using Nested for loop.

(1) *, **, *** ... (2) 5 5 5 5 5, 4 4 4 4 ...

Answer:

Nested for loop: A loop within another loop. The inner loop completes all its iterations for every single iteration of the outer loop.

Syntax:

```c
for (initialization; condition; increment) {
    for (initialization; condition; increment) {
        // statements
    }
}
```

C Program for Pattern 2 (Numbers):

```c
#include <stdio.h>
void main() {
    int i, j;
    for(i = 5; i >= 1; i--) {
        for(j = 1; j <= i; j++) {
            printf("%d ", i);
        }
        printf("\n");
    }
}
```

Question 4b

If a = 8, b = 15, and c = 4, evaluate the following expression:

(1) 2 * ((a % 5) * (4 + (b – 3) / (c + 2)))

(2) `a += b *= c -= 5` (Given a=3, b=5, c=8)

Answer:

- **Expression 1:**
  - $a\%5 = 8\%5 = 3$
  - $b-3 = 12$; $c+2 = 6$
  - $12 / 6 = 2$
  - $4 + 2 = 6$
  - $3 * 6 = 18$
  - $2 * 18 = 36$
  - **Result: 36**
- **Expression 2 (a=3, b=5, c=8):**
  - Evaluates right to left:
  - $c = 8 - 5 = 3$
  - $b = 5 * 3 = 15$
  - $a = 3 + 15 = 18$
  - **Result: a=18, b=15, c=3**

Question 5a

Explain for loop with syntax. Develop a C program to generate the Fibonacci Series using for loop.

Answer:

Syntax: `for(initialization; condition; increment/decrement) { statements; }`

Fibonacci Program:

```C
#include <stdio.h>
void main() {
    int n, i, f1 = 0, f2 = 1, f3;
    printf("Enter number of terms: ");
    scanf("%d", &n);
    printf("Series: %d %d ", f1, f2);
    for(i = 3; i <= n; i++) {
        f3 = f1 + f2;
        printf("%d ", f3);
        f1 = f2;
        f2 = f3;
    }
}
```

Question 5b

Differentiate between while loop and do-while loop with its syntax. Write a C program using a while loop to find the factorial of a given number.

Answer:

while loop	do-while loop
while(condition) { ... }	do { ... } while(condition);
Entry-controlled.	Exit-controlled.

Factorial using while loop:

```c
#include <stdio.h>
void main() {
    int n, fact = 1;
    printf("Enter a number: ");
    scanf("%d", &n);
    int i = n;
    while(i > 0) {
        fact *= i;
        i--;
    }
    printf("Factorial: %d", fact);
}
```

Question 6a

Develop a C program to find the reverse of a positive integer and check for Palindrome.

Answer:

```c
#include <stdio.h>
void main() {
    int n, reversed = 0, original, remainder;
    printf("Enter an integer: ");
    scanf("%d", &n);
    original = n;
    while (n != 0) {
        remainder = n % 10;
        reversed = reversed * 10 + remainder;
        n /= 10;
    }
    if (original == reversed)
        printf("%d is a palindrome.", original);
    else
        printf("%d is not.", original);
}
```

Question 6b

Explain any two Unconditional control transfer statements with syntax and example.

Answer:

1. **break:** Used to exit a loop or switch statement immediately.
    - **Syntax:** `break;`
    - **Example:** `for(i=1; i<=10; i++) { if(i==5) break; }` (stops loop when i is 5).
2. **continue:** Skips the current iteration of a loop and moves to the next one.
    - **Syntax:** `continue;`
    - **Example:** `for(i=1; i<=5; i++) { if(i==3) continue; printf("%d", i); }` (Prints 1, 2, 4, 5).

Based on the notes for Module 1 and Module 2, here are the solutions for the fourth question paper (IA1 QP CTP.docx) up to question 6b.

Question 1a

Discuss any four computational thinking key techniques. Identify the problem and find a solution for a manufacturing company experiencing a significant increase in product defects.

Answer:

The four key techniques of Computational Thinking are:

1. **Decomposition:** Breaking down a complex problem into smaller, more manageable parts.
2. **Pattern Recognition:** Looking for similarities or trends within problems.
3. **Abstraction:** Focusing on the important information only, ignoring irrelevant details.
4. **Algorithms:** Developing a step-by-step solution to the problem.

Manufacturing Problem Analysis:

- **Problem Identification:** Significant increase in product defects.
- **Solution:** Use **Decomposition** to check different stages of the production line (raw materials, machinery, human error). Use **Pattern Recognition** to see if defects occur at a specific time or on a specific machine. Apply **Abstraction** to focus on the defect rate rather than the product color. Finally, design an **Algorithm** (standard operating procedure) to calibrate machines and train staff to reduce errors.

Question 1b

List and define the purpose of any six symbols used in flowchart. Design a flowchart to find: (i) Smallest of two numbers (ii) Simple Interest.

Answer:

Flowchart Symbols:

1. **Oval (Terminal):** Indicates the Start or Stop of the process.
2. **Parallelogram (Input/Output):** Used for reading data or printing results.
3. **Rectangle (Process):** Used for arithmetic calculations or variable assignments.
4. **Diamond (Decision):** Used for "Yes/No" or "True/False" conditions.
5. **Arrows (Flow lines):** Shows the direction of the process flow.
6. **Circle (Connector):** Connects different parts of a flowchart.

Flowchart Designs:

- **(i) Smallest of two numbers:** Start $\rightarrow$ Input A, B $\rightarrow$ Decision (Is A < B?) $\rightarrow$ If Yes, Output A; If No, Output B $\rightarrow$ Stop.
- **(ii) Simple Interest:** Start $\rightarrow$ Input P, T, R $\rightarrow$ Process (SI = (P*T*R)/100) $\rightarrow$ Output SI $\rightarrow$ Stop.

Question 2a

Define computer. Explain the various parts of a digital computer with a neat block diagram.

Answer:

A **computer** is an electronic device that processes data according to a set of instructions to produce a meaningful output.

Parts of a Digital Computer:

1. **Input Unit:** Devices like keyboard and mouse used to enter data.
2. **Central Processing Unit (CPU):** The brain of the computer, consisting of:
   - **Control Unit (CU):** Manages and coordinates all components.
   - **Arithmetic Logic Unit (ALU):** Performs mathematical and logical operations.
3. **Memory Unit:** Stores data and instructions (RAM for temporary, Hard Drive for permanent).
4. **Output Unit:** Devices like monitors and printers that display the processed information.

Question 2b

Design an algorithm and flowchart to check whether a given number is prime or not.

Answer:

Algorithm:

1. Start.
2. Read number N.
3. If $N \le 1$, it is not prime.
4. For $i = 2$ to $\sqrt{N}$:
   - If $N \% i == 0$, then N is not prime, go to step 6.
5. If the loop completes, N is prime.
6. Print result and Stop.

Question 3a

List and explain different types of data types available in C language.

Answer:

1. **int:** Used for whole numbers (integers). Size: 2 or 4 bytes. (e.g., `int age = 25;`)

2. **float:** Used for numbers with decimal points. Size: 4 bytes. (e.g., `float price = 99.50;`)
3. **char:** Used for single characters. Size: 1 byte. (e.g., `char grade = 'A';`)
4. **double:** Used for large floating-point numbers with high precision. Size: 8 bytes.

Question 3b

Develop a C program to find the area and perimeter of a rectangle.

Answer:

```C
#include <stdio.h>
void main() {
    float l, w, area, peri;
    printf("Enter length and width: ");
    scanf("%f %f", &l, &w);
    area = l * w;
    peri = 2 * (l + w);
    printf("Area = %.2f, Perimeter = %.2f", area, peri);
}
```

Question 4a

Explain the basic structure of C program. List and explain any four types of operators.

Answer:

Structure: Link Section (#include) $\rightarrow$ Main Function (`void main()`) $\rightarrow$ Variable Declaration $\rightarrow$ Executable Statements.

Operators:

1. **Arithmetic:** `+`, `-`, `*`, `/` (Basic math).
2. **Relational:** `==`, `>`, `<` (Comparison).
3. **Logical:** `&&`, `||`, `!` (Combining conditions).
4. **Assignment:** `=` (Storing values).

Question 4b

Construct a C program to check whether a given year is a leap year or not using if-else ladder.

Answer:

```c
#include <stdio.h>
void main() {
    int year;
    printf("Enter year: ");
    scanf("%d", &year);
    if (year % 400 == 0) printf("Leap Year");
    else if (year % 100 == 0) printf("Not a Leap Year");
    else if (year % 4 == 0) printf("Leap Year");
    else printf("Not a Leap Year");
}
```

Question 5a

Differentiate between for loop and while loop. Write a C program to find the sum of first N natural numbers using while loop.

Answer:

- **for loop:** Used when the number of iterations is known in advance.
- **while loop:** Used when the number of iterations depends on a condition being true.

Program:

```c
#include <stdio.h>
void main() {
    int n, i = 1, sum = 0;
    printf("Enter N: ");
    scanf("%d", &n);
    while(i <= n) {
        sum += i;
        i++;
    }
    printf("Sum = %d", sum);
}
```

Question 5b

Develop a C program to check whether the given number is an Armstrong number or not

using do-while loop.

Answer:

```c
#include <stdio.h>
void main() {
    int n, original, rem, res = 0;
    printf("Enter integer: ");
    scanf("%d", &n);
    original = n;
    do {
        rem = n % 10;
        res += rem * rem * rem;
        n /= 10;
    } while (n != 0);
    if (res == original) printf("Armstrong");
    else printf("Not Armstrong");
}
```

Question 6a

Write a C program to print the following pattern.

(i) `*`, `**`, `***` ... (ii) `1`, `2 2`, `3 3 3` ...

Answer (Pattern ii):

```c
#include <stdio.h>
void main() {
    int i, j;
    for(i = 1; i <= 5; i++) {
        for(j = 1; j <= i; j++) {
            printf("%d ", i);
        }
        printf("\n");
    }
}
```

Question 6b

Write the output of the following programs.

Answer:

- **(i)** for(i=1; i<=10; i++) { if(i%5==0) break; printf("%d\t", i); }
    - **Output: 1 2 3 4**
- **(ii)** for(i=1; i<=10; i++) { if(i%5==0) continue; printf("%d\t", i); }
    - **Output: 1 2 3 4 6 7 8 9**
- **(iii)** printf("MITE\t"); goto down; printf("IS IN"); down: printf("MOODABIDRI");
    - **Output: MITE MOODABIDRI**
- **(iv)** switch(a) { case 5: printf("I am 5"); break; case 7: printf("I am 7"); break; } (Given a=5)
    - **Output: I am 5**

[2026-04-17 1:45 PM IST]

Based on the notes for Module 1 and Module 2, here are the solutions for the fourth question paper (IA1 QP CTP.docx) up to question 6b.

Question 1a

Discuss any four computational thinking key techniques. Identify the problem and find a solution for a manufacturing company experiencing a significant increase in product defects.

Answer:

The four key techniques of Computational Thinking are:

1. **Decomposition:** Breaking down a complex problem into smaller, more manageable parts.
2. **Pattern Recognition:** Looking for similarities or trends within problems.
3. **Abstraction:** Focusing on the important information only, ignoring irrelevant details.
4. **Algorithms:** Developing a step-by-step solution to the problem.

Manufacturing Problem Analysis:

- **Problem Identification:** Significant increase in product defects.
- **Solution:** Use **Decomposition** to check different stages of the production line (raw materials, machinery, human error). Use **Pattern Recognition** to see if defects occur at a specific time or on a specific machine. Apply **Abstraction** to focus on the defect rate rather than the product color. Finally, design an **Algorithm** (standard operating procedure) to calibrate machines and train staff to reduce errors.

Question 1b

List and define the purpose of any six symbols used in flowchart. Design a flowchart to find: (i) Smallest of two numbers (ii) Simple Interest.

Answer:

Flowchart Symbols:

1. **Oval (Terminal):** Indicates the Start or Stop of the process.
2. **Parallelogram (Input/Output):** Used for reading data or printing results.
3. **Rectangle (Process):** Used for arithmetic calculations or variable assignments.
4. **Diamond (Decision):** Used for "Yes/No" or "True/False" conditions.
5. **Arrows (Flow lines):** Shows the direction of the process flow.
6. **Circle (Connector):** Connects different parts of a flowchart.

Flowchart Designs:

- **(i) Smallest of two numbers:** Start $\rightarrow$ Input A, B $\rightarrow$ Decision (Is A < B?) $\rightarrow$ If Yes, Output A; If No, Output B $\rightarrow$ Stop.
- **(ii) Simple Interest:** Start $\rightarrow$ Input P, T, R $\rightarrow$ Process (SI = (P*T*R)/100) $\rightarrow$ Output SI $\rightarrow$ Stop.

Question 2a

Define computer. Explain the various parts of a digital computer with a neat block diagram.

Answer:

A **computer** is an electronic device that processes data according to a set of instructions to produce a meaningful output.

Parts of a Digital Computer:

1. **Input Unit:** Devices like keyboard and mouse used to enter data.
2. **Central Processing Unit (CPU):** The brain of the computer, consisting of:
   - **Control Unit (CU):** Manages and coordinates all components.
   - **Arithmetic Logic Unit (ALU):** Performs mathematical and logical operations.
3. **Memory Unit:** Stores data and instructions (RAM for temporary, Hard Drive for permanent).
4. **Output Unit:** Devices like monitors and printers that display the processed information.

Question 2b

Design an algorithm and flowchart to check whether a given number is prime or not.

Answer:

Algorithm:

1. Start.
2. Read number N.
3. If $N \le 1$, it is not prime.
4. For $i = 2$ to $\sqrt{N}$:
   - If $N \% i == 0$, then N is not prime, go to step 6.
5. If the loop completes, N is prime.
6. Print result and Stop.

Question 3a

List and explain different types of data types available in C language.

Answer:

1. **int:** Used for whole numbers (integers). Size: 2 or 4 bytes. (e.g., `int age = 25;`)
2. **float:** Used for numbers with decimal points. Size: 4 bytes. (e.g., `float price = 99.50;`)
3. **char:** Used for single characters. Size: 1 byte. (e.g., `char grade = 'A';`)
4. **double:** Used for large floating-point numbers with high precision. Size: 8 bytes.

Question 3b

Develop a C program to find the area and perimeter of a rectangle.

Answer:

C

```c
#include <stdio.h>
void main() {
    float l, w, area, peri;
    printf("Enter length and width: ");
    scanf("%f %f", &l, &w);
    area = l * w;
    peri = 2 * (l + w);
    printf("Area = %.2f, Perimeter = %.2f", area, peri);
}
```

Question 4a

Explain the basic structure of C program. List and explain any four types of operators.

Answer:

Structure: Link Section (#include) $\rightarrow$ Main Function (`void main()`) $\rightarrow$ Variable Declaration $\rightarrow$ Executable Statements.

Operators:

1. **Arithmetic:** `+`, `-`, `*`, `/` (Basic math).
2. **Relational:** `==`, `>`, `<` (Comparison).
3. **Logical:** `&&`, `||`, `!` (Combining conditions).
4. **Assignment:** `=` (Storing values).

Question 4b

Construct a C program to check whether a given year is a leap year or not using if-else ladder.

Answer:

C

```c
#include <stdio.h>
void main() {
    int year;
    printf("Enter year: ");
    scanf("%d", &year);
    if (year % 400 == 0) printf("Leap Year");
    else if (year % 100 == 0) printf("Not a Leap Year");
    else if (year % 4 == 0) printf("Leap Year");
    else printf("Not a Leap Year");
}
```

Question 5a

Differentiate between for loop and while loop. Write a C program to find the sum of first N natural numbers using while loop.

Answer:

- **for loop:** Used when the number of iterations is known in advance.
- **while loop:** Used when the number of iterations depends on a condition being true.

Program:

```c
#include <stdio.h>
void main() {
    int n, i = 1, sum = 0;
    printf("Enter N: ");
    scanf("%d", &n);
    while(i <= n) {
        sum += i;
        i++;
    }
    printf("Sum = %d", sum);
}
```

Question 5b

Develop a C program to check whether the given number is an Armstrong number or not using do-while loop.

Answer:

```c
#include <stdio.h>
void main() {
    int n, original, rem, res = 0;
    printf("Enter integer: ");
    scanf("%d", &n);
    original = n;
    do {
        rem = n % 10;
        res += rem * rem * rem;
        n /= 10;
    } while (n != 0);
    if (res == original) printf("Armstrong");
    else printf("Not Armstrong");
}
```

Question 6a

Write a C program to print the following pattern.

(i) *, **, *** ... (ii) 1, 2 2, 3 3 3 ...

Answer (Pattern ii):

```
C

#include <stdio.h>
void main() {
    int i, j;
    for(i = 1; i <= 5; i++) {
        for(j = 1; j <= i; j++) {
            printf("%d ", i);
        }
        printf("\n");
    }
}
```

Question 6b

Write the output of the following programs.

Answer:

- **(i)** `for(i=1; i<=10; i++) { if(i%5==0) break; printf("%d\t", i); }`
  - **Output: 1 2 3 4**
- **(ii)** `for(i=1; i<=10; i++) { if(i%5==0) continue; printf("%d\t", i); }`
  - **Output: 1 2 3 4 6 7 8 9**
- **(iii)** `printf("MITE\t"); goto down; printf("IS IN"); down: printf("MOODABIDRI");`
  - **Output: MITE MOODABIDRI**
- **(iv)** `switch(a) { case 5: printf("I am 5"); break; case 7: printf("I am 7"); break; }` (Given a=5)
  - **Output: I am 5**

[2026-04-17 1:45 PM IST]